

French Sentence Correction TIPE

A prototype tool to automatically correct simple French sentences by combining formal-grammar parsing with lexicon-based techniques. Built as a TIPE (Travail d'Initiative Personnelle Encadrée) project in theoretical computer science, it demonstrates core concepts in lexing, parsing, data structures and error-correction algorithms.

Table of Contents

- Motivation
 - Problem Statement
 - Objectives
 - Key Concepts
 - Features
 - Requirements
 - Installation
 - Usage
 - Project Structure
 - Architecture Overview
 - Bibliography
 - License
-

Motivation

During my second year of preparatory classes, I discovered formal language theory, which immediately fascinated me. I wanted to understand how orthographic and grammatical correctors transform incorrect French sentences into correct ones, and chose this topic for my TIPE.

Problem Statement

How can we build a program that automatically corrects French sentences?

We focus on simple sentences and a restricted subset of French grammar to illustrate core techniques without requiring large-scale NLP models.

Objectives

- **Lexical Analysis:** Tokenize input strings into grammatical classes.

- **Syntax Parsing:** Construct syntax trees conforming to a formal grammar.
 - **Error Detection & Correction:** Identify syntactic mistakes or typos, propose one or more corrected alternatives.
 - **User-Adapted Suggestions:** Rank corrections using Levenshtein distance and customizable word-frequency data.
-

Key Concepts

- **Formal Grammars & Parsing**
 - **Lexical Analysis (Lexing)**
 - **Syntax Trees**
 - **Trie Data Structure** for fast dictionary lookup
 - **Levenshtein Distance** for typo correction
 - **User-specific Word Frequencies**
-

Features

- Parses and corrects simple French phrases against a custom, non-context-free grammar.
 - Supports multiple token interpretations per word (e.g., homonyms).
 - Stores dictionary in a Trie for sub-linear lookup.
 - Generates and ranks multiple correction proposals.
 - Allows integration of user-specific frequency lists.
-

Requirements

- **OCaml**
 - **dune** build system
 - Unix-compatible environment
-

Installation

```
# Clone the repository
git clone https://github.com/akSkwYX/french-sentence-correction-tipe.git
cd french-sentence-correction-tipe

# Build the project
dune build

# Optionally install locally
```

Usage

```
# Run the correction tool on an input sentence
dune exec main -- "Le petite chat rouge bois du lait"

# Sample output:
# Le petit chat rouge boit du lait
```

Project Structure

```
|-- bin/
|   |-- corrector.ml      # Main executable
|-- lib/
|   |-- lexer.ml         # Lexical analysis
|   |-- parser.ml        # Grammar parser
|   |-- tree.ml          # Syntax tree definitions
|   |-- corrector.ml     # Verification & correction logic
|-- data/
|   |-- french_grammar.cfg # Grammar rules
|   |-- dictionary.trie   # Serialized Trie of allowed words
|   |-- frequency.json    # Word-frequency data
|-- test/
|   |-- test_corrector.ml # Unit tests
|-- mcot.md              # Project summary and bibliographic comments
|-- presentation.tex     # TIPE presentation slides
\-- dune-project
```

Architecture Overview

1. Lexing

- Reads an input string and produces a list of possible token sequences
- Handles multiple grammatical classes for ambiguous words

2. Parsing

- Uses an LL/LR parser to build syntax trees from tokens
- Grammar defined as a quadruple (V, T, R, S) over a restricted French subset

3. Verification & Correction

- Traverses the syntax tree to detect syntactic violations

- Applies Levenshtein distance to suggest typographical fixes
- Ranks results using user-specific frequency weights

4. Data Storage

- Dictionary stored in a memory-optimized Trie (height ~ 39 , size $\sim 983\,809$ entries)

Bibliography

1. Hopcroft, J.E. & Ullman, J.D. *Introduction to Automata Theory, Languages, and Computation*.
2. Fredkin, E. “Trie Memory.” *Commun. ACM* 3, no.9 (1960): 490–499.
3. Grammalecte. *Open-source French grammar checker*. <https://github.com/Pofilo/grammalecte>
4. Levenshtein, V.I. “Binary codes capable of correcting deletions, insertions, and reversals.” (1966).